

LAB NOTE · RL RUNNER BENCHMARK

PPO CartPole-v1, GPU 가 CPU보다 2.5배 느렸던 이유

강화학습을 처음 돌려보면서 러너(runner) 구조와 GPU 활용을 실측으로 확인한 기록. 연구실 워크스테이션에서 재현 가능.

날짜 2026-09-15 GPU RTX PRO 6000 Blackwell Max-Q (98GB) CPU 32 logical cores

환경 conda RL_Book · Python 3.9.25 · torch 2.8.0+cu128

01 배경 — 왜 확인했나

PPO로 CartPole-v1을 학습시키던 중 GPU를 "풀로" 쓰고 있는지 궁금해서 `nvidia-smi dmon`으로 학습 중 사용률을 실측했다. 결과는 예상 밖이었다 — GPU 사용률이 10%도 안 됐고, 같은 학습을 CPU로 돌리니 오히려 더 빨랐다. 아래는 그 과정과 수치, 그리고 연구실 환경에 적용할 결론이다.

02 실험 환경

GPU RTX PRO 6000 Blackwell Max-Q	VRAM 97,887 MiB	CPU 32 logical cores
실행 환경 conda env "RL_Book" (Py 3.9.25)	주요 패키지 torch 2.8.0+cu128 · ray 1.13.0	알고리즘 / 환경 PPO / CartPole-v1 (n_envs=1)

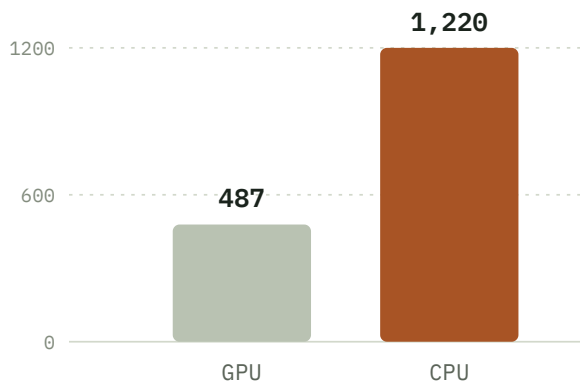
03 Runner 구조 — 3가지 실행 방식

이 프레임워크는 `n_envs`와 `distributed_processing_type` 설정으로 아래 세 러너 중 하나가 선택된다.

러너	조건	특징
Runner	n_envs = 1	환경 1개, 단일 프로세스 순차 실행. 분산 처리 오버헤드 없음. 지금 이 벤치마크가 사용한 방식.
MultiEnvRunner	n_envs > 1, sync	Ray로 환경을 여러 프로세스에 분산. 매 라운드마다 전체 환경이 끝날 때까지 대기 후 학습 — 가장 느린 환경이 전체를 지연시킴.
MultiEnvAsyncRunner	n_envs > 1, async	Ray로 분산하되 먼저 끝난 환경 결과부터 즉시 학습에 반영. 처리량은 높지만 정책 스냅샷이 환경마다 살짝 어긋날 수 있음.

04 실측 결과

100,000 스텝 소요 시간 · STEPS/SEC



CPU가 GPU보다

2.5x

빠름 (steps/sec 기준). CartPole은 상태가 작고(float 4개) 네트워크도 작아, 매 스텝 GPU 왕복이 손해였다.

학습 중 GPU 실사용률 (NVIDIA-SMI DMON, 240초 샘플)



VRAM 707MB / 97,887MB · 전력 64~73W / 300W. GPU는 대부분 대기 상태였다.

05 원인

agents/actor.py의 select_action()이 매 스텝마다 상태 1개(batch=1)를 CPU→GPU로 보내 forward 하고 다시 CPU로 받아온다(runner/environment_loop.py의 while 루프에서 스텝마다 반복). 이 왕복 오버헤드(커널 실행 지연 + PCIe 전송)가 512유닛 MLP 한 번의 실제 연산량보다 커서, GPU는 대부분 다음 호출을 기다리며 놀았다.

06 **n_envs=4/16 × sync/async × CPU/GPU 전수 비교** (09-16 추가)

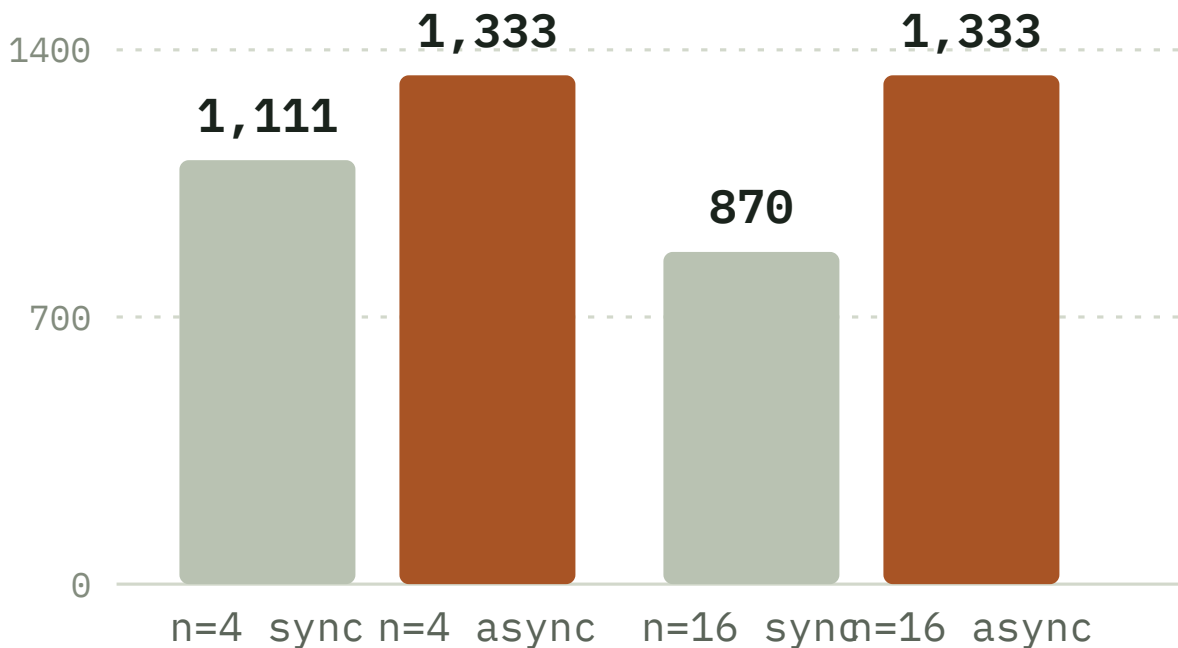
MultiEnvRunner(sync)와 MultiEnvAsyncRunner(async)를 두 환경 모두에서 n_envs=4/16(32코어 중 더 활용), CPU/GPU 각각 실측했다. 저장돼 있던 *-4-sync.yaml 변형 파일은 CLI로 바로 실행이 안 돼서(-e 인자가 내부 env_name을 덮어써 gym.error.NameNotFound 발생), 매번 본 config를 고쳐 실행 → 측정 → git checkout으로 원복하는 방식으로 진행했다.

환경	N_ENVS	처리방식	GPU	STEPS/SEC
CartPole-v1	1	–	CPU	1,220
CartPole-v1	1	–	GPU	487
CartPole-v1	4	sync	CPU	1,724
CartPole-v1	4	async	CPU	2,000
CartPole-v1	4	sync	GPU	556
CartPole-v1	4	async	GPU	806
CartPole-v1	16	sync	CPU	2,000
CartPole-v1	16	async	CPU	1,923
CartPole-v1	16	sync	GPU	559
CartPole-v1	16	async	GPU	794
AntBulletEnv-v0	1	–	CPU	625
AntBulletEnv-v0	1	–	GPU	323
AntBulletEnv-v0	4	sync	CPU	1,111
AntBulletEnv-v0	4	async	CPU	1,333
AntBulletEnv-v0	4	sync	GPU	500
AntBulletEnv-v0	4	async	GPU	571
AntBulletEnv-v0	16	sync	CPU	870

환경	N_ENVS	처리방식	GPU	STEPS/SEC
AntBulletEnv-v0	16	async	CPU	1,333
AntBulletEnv-v0	16	sync	GPU	328
AntBulletEnv-v0	16	async	GPU	455

전체 20,000 스텝(원래 3,000,000에서 벤치마크용으로 축소) 기준 · AntBulletEnv-v0

ANTBULLETV0, CPU — 워커가 늘수록 SYNC는 느려지고 ASYNC는 그대로



핵심 발견

- **모든 조합에서 CPU가 GPU를 이긴다** — 두 환경, 모든 n_envs, sync/async 무관하게 일관됨.
- **async는 sync보다 항상 같거나 빠르다.** AntBulletEnv에서 n_envs=16 sync(870)가 n_envs=4 sync(1,111)보다 오히려 느려짐 — 워커가 늘수록 straggler 효과가 커지는 것. async는 4→16에서도 1,333으로 그대로 — **스케일에 더 견고함.**
- **4→16 확장은 이득이 없거나 손해.** 두 환경 모두 CPU async 기준 n_envs=4에서 이미 포화. GPU는 16에서 더 느려지기도 함(프로세스들이 물리 GPU 1개를 두고 경쟁하는 것으로 추정).
- **실측 최적 설정: use_cuda: False, n_envs=4, async.** 두 환경 모두에서 가장 빨랐고, 16까지 늘리는 건 이 실험에서는 이득이 없었다.

07 연구실 환경 적용 가이드

결론: "GPU를 켜다" ≠ "GPU를 효율적으로 쓴다", "환경을 늘렸다" ≠ "더 빨라진다"

- **가벼운 환경** (CartPole류, 저차원 상태, 작은 MLP) → use_cuda: False, n_envs=4, async가 실측상 가장 빠름(2,000 steps/sec)이지만, 구현 단순성이 더 중요하다면 n_envs=1도 충분.
- **무거운 환경** (AntBulletEnv류 물리 시뮬레이션) → 역시 use_cuda: False + n_envs=4, async가 실측 최적. 이미지 관측처럼 네트워크 자체가 커지는 경우에만 GPU 재검토.
- **병렬 규모는 무조건 늘린다고 좋아지지 않는다.** sync는 워커가 늘수록 느려질 수 있고, async도 특정 지점(이 실험에선 n_envs=4)에서 포화된다. 늘리기 전에 항상 실측할 것.
- 결정 전에 항상 nvidia-smi dmon -s pucm -d 1로 학습 중 SM 사용률을 실측해서 병목이 CPU(환경)인지 GPU(연산)인지 먼저 확인한다.

08 재현 방법

```
# RL_Book conda 환경으로 실행 (시스템 python엔 ray/tensorboard 없음)
"C:/Users/User/WorkSpace/Donghun/miniconda3/envs/RL_Book/python.exe" main.py -a ppo -e CartPole

# 학습 중 GPU 사용률 실측
nvidia-smi dmon -s pucm -d 1 -o T
```

ENVIRONMENT.md · pytorch_RL 저장소 기록 참고 — 다음 비교 예정: 더 큰 네트워크에서 GPU가 유리해지는 지점, AntBulletEnv 3,000,000 스텝 완주 시간